

# Econometrics Stata Lab: Merging and Reshaping Data

Suraj Kumar

September 18, 2026

*Examples adapted from the UCLA Statistical Methods and Data Analytics Stata Learning Modules.*

*Prepared with assistance from OpenAI GPT-5.6 Sol.*

# Learning goals

---

By the end of this lecture, you should be able to:

1. identify the **unit of observation** and the **key** of a dataset;
2. distinguish a **1:1 merge** from a **1:m merge**;
3. use `_merge` to check whether observations matched;
4. distinguish **wide** data from **long** data; and
5. explain what `i()` and `j()` do in a reshape command.

# Begin with two questions

---

Before merging or reshaping, ask:

1. **What does one row represent?**

A person, family, school, firm, country-year, or something else?

2. **Which variable or variables uniquely identify a row?**

This is the dataset's **key**.

Examples:

- One row per family: key is famid.
- One row per child: key may be famid birth.
- One row per school-year: key may be schid year.

In Stata, use `isid key_variables` to check the proposed key.

# What does merging do?

---

**Merging adds columns** from one dataset to matching rows in another dataset.

Stata uses two terms:

- **Master data**: the dataset currently in memory after use.
- **Using data**: the dataset named after using.

A merge needs:

- a common key in both files; and
- the correct statement about whether that key appears once or many times.

**The numbers in 1:1 and 1:m describe the key, not the number of files.**

## One-to-one merge in motion

---

A **1:1 merge** is appropriate when the key appears at most once in each file.

| Dads |      |      |
|------|------|------|
| ID   | Name | Inc. |
| 1    | Bill | 30   |
| 2    | Art  | 22   |
| 3    | Paul | 25   |

| Family income |    |      |      |
|---------------|----|------|------|
| ID            | 96 | 97   | 98   |
| 1             | 40 | 40.5 | 41   |
| 2             | 45 | 45.4 | 45.8 |
| 3             | 75 | 76   | 77   |

| Merged |      |      |    |
|--------|------|------|----|
| ID     | Name | Inc. | 96 |
|        |      |      |    |
|        |      |      |    |
|        |      |      |    |

```
merge 1:1 famid using faminc.dta
```

Stata begins with the key famid in both files.

# One-to-one merge in motion

A **1:1 merge** is appropriate when the key appears at most once in each file.

| Dads |      |      | Family income |    |      |      | Merged |      |      |    |
|------|------|------|---------------|----|------|------|--------|------|------|----|
| ID   | Name | Inc. | ID            | 96 | 97   | 98   | ID     | Name | Inc. | 96 |
| 1    | Bill | 30   | 1             | 40 | 40.5 | 41   | 1      | Bill | 30   | 40 |
| 2    | Art  | 22   | 2             | 45 | 45.4 | 45.8 |        |      |      |    |
| 3    | Paul | 25   | 3             | 75 | 76   | 77   |        |      |      |    |

`merge 1:1 famid using faminc.dta`

**Step 1:** famid=1 matches Bill to family 1's income.

# One-to-one merge in motion

A **1:1 merge** is appropriate when the key appears at most once in each file.

| Dads |      |      | Family income |    |      |      | Merged |      |      |    |
|------|------|------|---------------|----|------|------|--------|------|------|----|
| ID   | Name | Inc. | ID            | 96 | 97   | 98   | ID     | Name | Inc. | 96 |
| 1    | Bill | 30   | 1             | 40 | 40.5 | 41   | 1      | Bill | 30   | 40 |
| 2    | Art  | 22   | 2             | 45 | 45.4 | 45.8 | 2      | Art  | 22   | 45 |
| 3    | Paul | 25   | 3             | 75 | 76   | 77   |        |      |      |    |

```
merge 1:1 famid using faminc.dta
```

**Step 2:** famid=2 matches Art to family 2's income.

# One-to-one merge in motion

A **1:1 merge** is appropriate when the key appears at most once in each file.

| Dads |      |      | Family income |    |      |      | Merged |      |      |    |
|------|------|------|---------------|----|------|------|--------|------|------|----|
| ID   | Name | Inc. | ID            | 96 | 97   | 98   | ID     | Name | Inc. | 96 |
| 1    | Bill | 30   | 1             | 40 | 40.5 | 41   | 1      | Bill | 30   | 40 |
| 2    | Art  | 22   | 2             | 45 | 45.4 | 45.8 | 2      | Art  | 22   | 45 |
| 3    | Paul | 25   | 3             | 75 | 76   | 77   | 3      | Paul | 25   | 75 |

```
merge 1:1 famid using faminc.dta
```

**Step 3:** famid=3 matches Paul to family 3's income.

## Before a 1:1 merge

---

Check that the key is unique in **both** files:

```
use dads.dta, clear
```

```
isid famid
```

```
use faminc.dta, clear
```

```
isid famid
```

If either `isid` fails, do not immediately delete duplicates. First ask:

- Is the proposed key incomplete?
- Does one row represent a different unit than I thought?
- Are there genuine duplicate records or data errors?

## One-to-many merge in motion

---

A **1:m merge** is appropriate when the key appears once in the master data and several times in the using data.

| Dads (1) |      |
|----------|------|
| ID       | name |
| 1        | Bill |

| Children (m) |       |       |
|--------------|-------|-------|
| ID           | birth | child |
| 1            | 1     | Beth  |
| 1            | 2     | Bob   |
| 1            | 3     | Barb  |

| Merged |      |       |       |
|--------|------|-------|-------|
| ID     | name | birth | child |
|        |      |       |       |
|        |      |       |       |
|        |      |       |       |

```
merge 1:m famid using kids.dta
```

The master has one Bill row; the using data have three rows with famid=1.

## One-to-many merge in motion

A **1:m merge** is appropriate when the key appears once in the master data and several times in the using data.

| Dads (1) |      | Children (m) |       |       | Merged |      |       |       |
|----------|------|--------------|-------|-------|--------|------|-------|-------|
| ID       | name | ID           | birth | child | ID     | name | birth | child |
| 1        | Bill | 1            | 1     | Beth  | 1      | Bill | 1     | Beth  |
|          |      | 1            | 2     | Bob   |        |      |       |       |
|          |      | 1            | 3     | Barb  |        |      |       |       |

```
merge 1:m famid using kids.dta
```

**Step 1:** Bill matches Beth, producing the first child-level row.

## One-to-many merge in motion

A **1:m merge** is appropriate when the key appears once in the master data and several times in the using data.

| Dads (1) |      | Children (m) |       |       | Merged |      |       |       |
|----------|------|--------------|-------|-------|--------|------|-------|-------|
| ID       | name | ID           | birth | child | ID     | name | birth | child |
| 1        | Bill | 1            | 1     | Beth  | 1      | Bill | 1     | Beth  |
|          |      | 1            | 2     | Bob   | 1      | Bill | 2     | Bob   |
|          |      | 1            | 3     | Barb  |        |      |       |       |

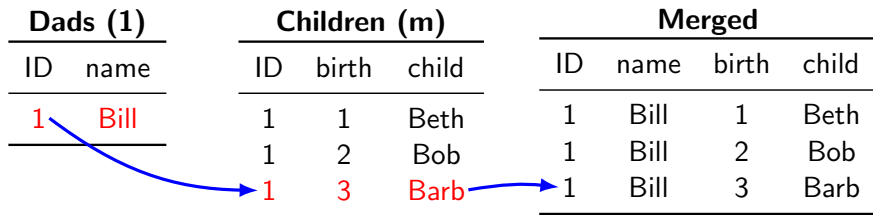
merge 1:m famid using kids.dta

**Step 2:** The same Bill row matches Bob and is repeated.

## One-to-many merge in motion

A **1:m merge** is appropriate when the key appears once in the master data and several times in the using data.

| Dads (1) |      | Children (m) |       |       | Merged |      |       |       |
|----------|------|--------------|-------|-------|--------|------|-------|-------|
| ID       | name | ID           | birth | child | ID     | name | birth | child |
| 1        | Bill | 1            | 1     | Beth  | 1      | Bill | 1     | Beth  |
|          |      | 1            | 2     | Bob   | 1      | Bill | 2     | Bob   |
|          |      | 1            | 3     | Barb  | 1      | Bill | 3     | Barb  |



```
merge 1:m famid using kids.dta
```

**Step 3:** Bill also matches Barb, producing three rows in total.

## Reading 1:m correctly

---

In merge 1:m famid using kids.dta:

- **1** means famid must be unique in the master data.
- **m** means famid may repeat in the using data.

Check the keys before merging:

- Dads: isid famid
- Children: isid famid birth

If the many-side file is already in memory, reverse the declaration:

```
merge m:1 famid using dads.dta
```

Avoid merge m:m. It usually creates unintended combinations rather than solving a key problem.

## Always inspect the merge result

---

Stata creates `_merge` after a merge.

| <code>_merge</code> | Meaning                                      |
|---------------------|----------------------------------------------|
| 1                   | Observation appears only in the master data. |
| 2                   | Observation appears only in the using data.  |
| 3                   | Observation matched in both datasets.        |

Useful checks:

- `tabulate _merge`
- `list if _merge != 3`
- `assert _merge == 3` when every row is expected to match

Do not drop `_merge` until you understand the unmatched observations.

# Merging versus reshaping

---

These operations solve different problems:

|              | Merge                                             | Reshape                                                    |
|--------------|---------------------------------------------------|------------------------------------------------------------|
| Purpose      | Combine information from different files          | Rearrange information within one dataset                   |
| Main effect  | Usually adds columns and sometimes rows           | Changes which information is stored in rows versus columns |
| Key question | How often does the merge key appear in each file? | What identifies a unit, and what varies within that unit?  |

## Wide to long in motion: one row becomes three

---

```
reshape long faminc, i(famid) j(year)
```

**Wide: one family row**

| famid | inc96 | inc97 | inc98 |
|-------|-------|-------|-------|
| 1     | 40000 | 40500 | 41000 |

**Long: three family-year rows**

| famid | year | faminc |
|-------|------|--------|
|-------|------|--------|

---

One wide observation stores three years in three columns.

## Wide to long in motion: one row becomes three

---

```
reshape long faminc, i(famid) j(year)
```

| Wide: one family row |       |       |       |
|----------------------|-------|-------|-------|
| famid                | inc96 | inc97 | inc98 |
| 1                    | 40000 | 40500 | 41000 |

| Long: three family-year rows |      |        |
|------------------------------|------|--------|
| famid                        | year | faminc |
| 1                            | 96   | 40000  |

**Step 1:** The suffix 96 becomes year=96; its value moves into faminc.

## Wide to long in motion: one row becomes three

```
reshape long faminc, i(famid) j(year)
```

| Wide: one family row |       |       |       |
|----------------------|-------|-------|-------|
| famid                | inc96 | inc97 | inc98 |
| 1                    | 40000 | 40500 | 41000 |

| Long: three family-year rows |      |        |
|------------------------------|------|--------|
| famid                        | year | faminc |
| 1                            | 96   | 40000  |
| 1                            | 97   | 40500  |

**Step 2:** The suffix 97 creates a second observation for the same family.

## Wide to long in motion: one row becomes three

```
reshape long faminc, i(famid) j(year)
```

| Wide: one family row |       |       |       |
|----------------------|-------|-------|-------|
| famid                | inc96 | inc97 | inc98 |
| 1                    | 40000 | 40500 | 41000 |

| Long: three family-year rows |      |        |
|------------------------------|------|--------|
| famid                        | year | faminc |
| 1                            | 96   | 40000  |
| 1                            | 97   | 40500  |
| 1                            | 98   | 41000  |

**Step 3:** The suffix 98 creates the third observation. The new key is famid year.

## Long to wide in motion: three rows become one

---

```
reshape wide faminc, i(famid) j(year)
```

**Long: three family-year**

| rows  |      |        |
|-------|------|--------|
| famid | year | faminc |
| 1     | 96   | 40000  |
| 1     | 97   | 40500  |
| 1     | 98   | 41000  |

**Wide: one family row**

| famid | faminc96 | faminc97 | faminc98 |
|-------|----------|----------|----------|
| 1     |          |          |          |

The long data contain three observations for family 1.

## Long to wide in motion: three rows become one

```
reshape wide faminc, i(famid) j(year)
```

**Long: three family-year**

| rows  |      |        |
|-------|------|--------|
| famid | year | faminc |
| 1     | 96   | 40000  |
| 1     | 97   | 40500  |
| 1     | 98   | 41000  |

**Wide: one family row**

| famid | faminc96 | faminc97 | faminc98 |
|-------|----------|----------|----------|
| 1     | 40000    |          |          |

**Step 1:** year=96 becomes the suffix in faminc96.

## Long to wide in motion: three rows become one

---

reshape wide faminc, i(famid) j(year)

**Long: three family-year**

| rows  |      |        |
|-------|------|--------|
| famid | year | faminc |
| 1     | 96   | 40000  |
| 1     | 97   | 40500  |
| 1     | 98   | 41000  |

**Wide: one family row**

| famid | faminc96 | faminc97 | faminc98 |
|-------|----------|----------|----------|
| 1     | 40000    | 40500    |          |

**Step 2:** year=97 supplies faminc97.

## Long to wide in motion: three rows become one

---

reshape wide faminc, i(famid) j(year)

**Long: three family-year**

| rows  |      |        |
|-------|------|--------|
| famid | year | faminc |
| 1     | 96   | 40000  |
| 1     | 97   | 40500  |
| 1     | 98   | 41000  |

**Wide: one family row**

| famid | faminc96 | faminc97 | faminc98 |
|-------|----------|----------|----------|
| 1     | 40000    | 40500    | 41000    |

**Step 3:** year=98 supplies faminc98. The resulting key is famid.

## Wide to long: what changes?

---

```
reshape long faminc, i(famid) j(year)
```

Read the command from left to right:

- `reshape long`: change from wide to long.
- `faminc`: common stem of `faminc96`, `faminc97`, and `faminc98`.
- `i(famid)`: `famid` uniquely identifies a row before reshaping.
- `j(year)`: create `year` from the suffixes 96, 97, and 98.

Key before: `famid`

Key after: `famid year`

Use long form when repeated measurements should appear in separate rows, as in most panel-data work.

## Long to wide: what changes?

---

```
reshape wide age, i(famid) j(birth)
```

Read the command from left to right:

- `reshape wide`: change from long to wide.
- `age`: values in this column will be spread across new columns.
- `i(famid)`: create one resulting row for each family.
- `j(birth)`: use birth-order values 1, 2, and 3 as suffixes.

Key before: `famid birth`

Key after: `famid`

The result contains `age1`, `age2`, and `age3`.

# The reshape rule to remember

---

## Wide to long

The suffix moves **out of variable names** and becomes a new `j()` variable.

## Long to wide

The values of the `j()` variable move **into variable names** as suffixes.

Character suffixes require the `string` option:

- `reshape long name inc, i(famid) j(dadmom) string`
- `reshape wide name inc, i(famid) j(dadmom) string`

## A safe workflow

---

1. State what one row represents.
2. Identify and verify the key with `isid`.
3. Decide whether you need to combine files or rearrange one file.
4. Run `merge` or `reshape`.
5. Verify the new key with `isid`.
6. Check the number of observations and inspect the result with `list`.
7. For merges, inspect `_merge` before dropping it.

**Never choose a merge or reshape command before understanding the unit of observation.**

# Now demonstrate the ideas in Stata

---

Run these files in order:

1. `combining_data_ucla.do`
  - `append`, `merge 1:1`, `merge 1:m`, and `_merge`
2. `reshaping_wide_to_long_ucla.do`
  - numeric suffixes, multiple identifiers, multiple stems, and string suffixes
3. `reshaping_long_to_wide_ucla.do`
  - one variable, multiple variables, and string suffixes

## UCLA Statistical Methods and Data Analytics: Stata Learning Modules

- **Combining data**

<https://stats.oarc.ucla.edu/stata/modules/combining-data/>

- **Reshaping data wide to long**

<https://stats.oarc.ucla.edu/stata/modules/reshaping-data-wide-to-long/>

- **Reshaping data long to wide**

<https://stats.oarc.ucla.edu/stata/modules/reshaping-data-long-to-wide/>

# Tell me how the course is going

---

- Anonymous, about 1 minute, all questions optional.
- Pick the week, your section, and rate the pace.
- Fill it in once for each week, if you like.

[Open the feedback form](#)



Scan with your phone camera